

BBM418 - Assignment 3

Berk Bubuş - 21945939

May 5, 2024

1 Dataset

First of all, I used Google Colab to use all the data that is given.

```
1 from google.colab import drive
2 drive.mount('/content/drive/')
3 os.chdir('/content/drive/MyDrive/vision/raw-img')
4
5 dataset_folder = '/content/drive/MyDrive/vision/raw-img'
6 classes = os.listdir(dataset_folder)
7 data = []
8 limit = 0
```

Here I have data array and limit variable which is an integer. If it equals 0, that means our code will use all the data we give. If we give it a number rather than 0, for example, 10, it will take 10 of every class.

```
1 for i, c in enumerate(classes):
2     files = os.listdir(os.path.join(dataset_folder, c))
3     a=0
4     for f in files:
5         if limit == 0:
6             data_image = cv2.imread(os.path.join(dataset_folder, c, f))
7             data.append((data_image, i))
8         else :
9             a+=1
10            if a <= limit:
11                data_image = cv2.imread(os.path.join(dataset_folder, c, f))
12                data.append((data_image, i))
13 print(len(data))
```

Here we read all the data with its label and append it to the data array.

```
1 random.shuffle(data)
2 train_split = 0.7
3 val_split = 0.10
4 test_split = 0.20
5 train_data = data[:int(len(data)*train_split)]
6 val_data = data[int(len(data)*train_split):int(len(data)*(train_split+val_split))]
7 test_data = data[int(len(data)*(train_split+val_split)):]
```

Here we random.shuffle the data and split it to train(0.7), val(0.1), test(0.2)

```
1 batch_size = 32
2
3 train_data = [(cv2.resize(x, (300, 300)), y) for x, y in train_data]
4 val_data = [(cv2.resize(x, (300, 300)), y) for x, y in val_data]
5 test_data = [(cv2.resize(x, (300, 300)), y) for x, y in test_data]
6
7 transform = transforms.Compose([transforms.ToTensor(),
8 transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
9 train_data = [(transform(x), y) for x, y in train_data]
10 val_data = [(transform(x), y) for x, y in val_data]
11 test_data = [(transform(x), y) for x, y in test_data]
12
13 train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=False)
14 val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)
15 test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False)
```

Here we transform Tensor and normalize our data. and resize them to 300x300 all our images. We have batch size variable which is 32.

2 Part 1 - Modeling and Training a CNN classifier from Scratch

2.1 Some Parametric Details

```
1 self.conv1 = Conv2d(in_channels=numChannels, out_channels=16, kernel_size=(5, 5))
2 self.relu1 = ReLU()
3 self.maxpool1 = MaxPool2d(kernel_size=(2, 2), stride=(2, 2))
4 self.conv2 = Conv2d(in_channels=16, out_channels=32, kernel_size=(5, 5))
5 self.relu2 = ReLU()
6 self.maxpool2 = MaxPool2d(kernel_size=(2, 2), stride=(2, 2))
7 self.conv3 = Conv2d(in_channels=32, out_channels=64, kernel_size=(5, 5))
8 self.relu3 = ReLU()
9 self.maxpool3 = MaxPool2d(kernel_size=(2, 2), stride=(2, 2))
10 self.conv4 = Conv2d(in_channels=64, out_channels=128, kernel_size=(5, 5))
11 self.relu4 = ReLU()
12 self.maxpool4 = MaxPool2d(kernel_size=(2, 2), stride=(2, 2))
13 self.conv5 = Conv2d(in_channels=128, out_channels=256, kernel_size=(5, 5))
14 self.relu5 = ReLU()
15 self.conv6 = Conv2d(in_channels=256, out_channels=512, kernel_size=(5, 5))
16 self.relu6 = ReLU()
17 self.fc1 = Linear(in_features=25088, out_features=500)
18 self.relu7 = ReLU()
19 self.fc2 = Linear(in_features=500, out_features=classes)
20 self.logSoftmax = LogSoftmax(dim=1)
21
22 self.dropout = nn.Dropout(0.8)
```

Here we have 6 conv layers and 2 fc layers as given in the pdf. But I have more than 1 pooling layer because I had to minimize my in features at fc layer. Also, dropout gives max performance at 0.8.

```
1 model = CNN(3, len(classes))
2 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
3 model.to(device)
4
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.0004, momentum=0.9)
6 early_stopping = 5
7 criterion = nn.CrossEntropyLoss()
```

I used SGD as optimizer but I tried Adam first. Adam gave too bad performance so I changed to SGD. As criterion I used CrossEntropyLoss. Also I have early stopping which is 5. All the trains start with 100 epochs but If the minimum criterion of our train is 5 epochs behind, it breaks and helps to save time.

2.2 Convolution and Fully-Connected Layers

```
1
2 def forward(self, x):
3     x = self.conv1(x)
4     x = self.relu1(x)
5     x = self.maxpool1(x)
6     x = self.conv2(x)
7     x = self.relu2(x)
8     x = self.maxpool2(x)
9     x = self.conv3(x)
10    x = self.relu3(x)
11    x = self.maxpool3(x)
12    x = self.conv4(x)
13    x = self.relu4(x)
14    x = self.maxpool4(x)
15    x = self.conv5(x)
16    x = self.relu5(x)
17    x = self.conv6(x)
18    x = self.relu6(x)
19    x = flatten(x, 1)
```

```

20 x = self.fc1(x)
21 x = self.relu7(x)
22 x = self.dropout(x)
23 x = self.fc2(x)
24 output = self.logSoftmax(x)
25 return output

```

Actually, I explained most of this part in the previous one. In Addition, dropout works after fc1.

2.3 A table of train and tests

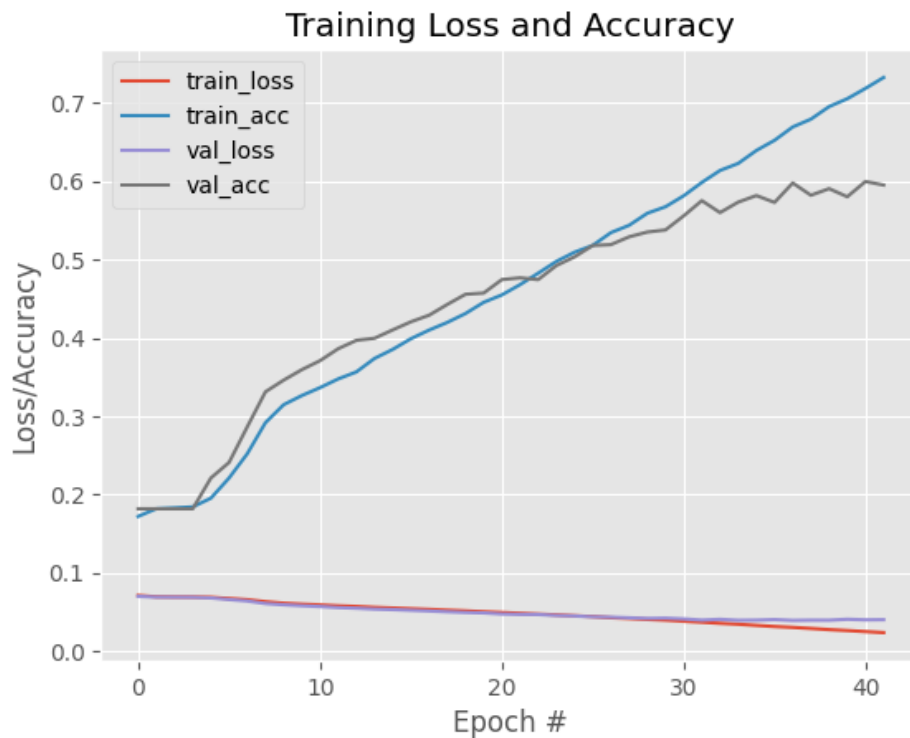
Trains and Test Results					
lr	early stop- ping	epoch break num- ber of 100	dropout	validation accu- racy	test accu- racy
0.0005	5	25	no dropout	0.4794	0.4851
0.0005	10	21	no dropout	0.4267	0.4215
0.0003	10	25	no dropout	0.4251	0.4312
0.0007	5	20	no dropout	0.4828	0.4788
0.0007	10	19	no dropout	0.4576	0.4597
0.0004	5	27	no dropout	0.4889	0.5023
0.0004	5	24	0.1	0.4500	0.4509
0.0004	5	21	0.2	0.4969	0.4977
0.0004	5	28	0.5	0.5332	0.5315
0.0004	100	100	0.5	0.5149	0.4691
0.0004	5	42	0.8	0.5978	0.5773
0.0002	5	31	no dropout	0.4186	0.4209

According to this table and our model:

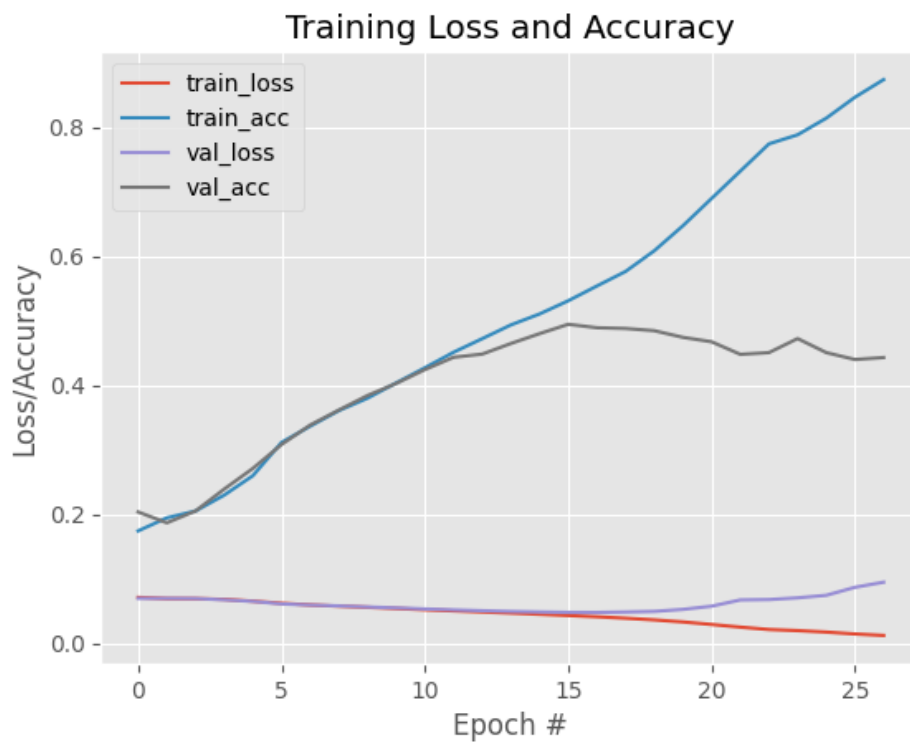
- Our best result came from 0.0004 with 0.8 dropout.
- Also, we can see that if we don't use early-stopping, our model overfits and gives lower accuracy.
- For early-stopping, 5 is better than 10.
- Dropout is not always good but 0.5 and 0.8 increases our models performance

2.4 Graph and Convolution Matrix

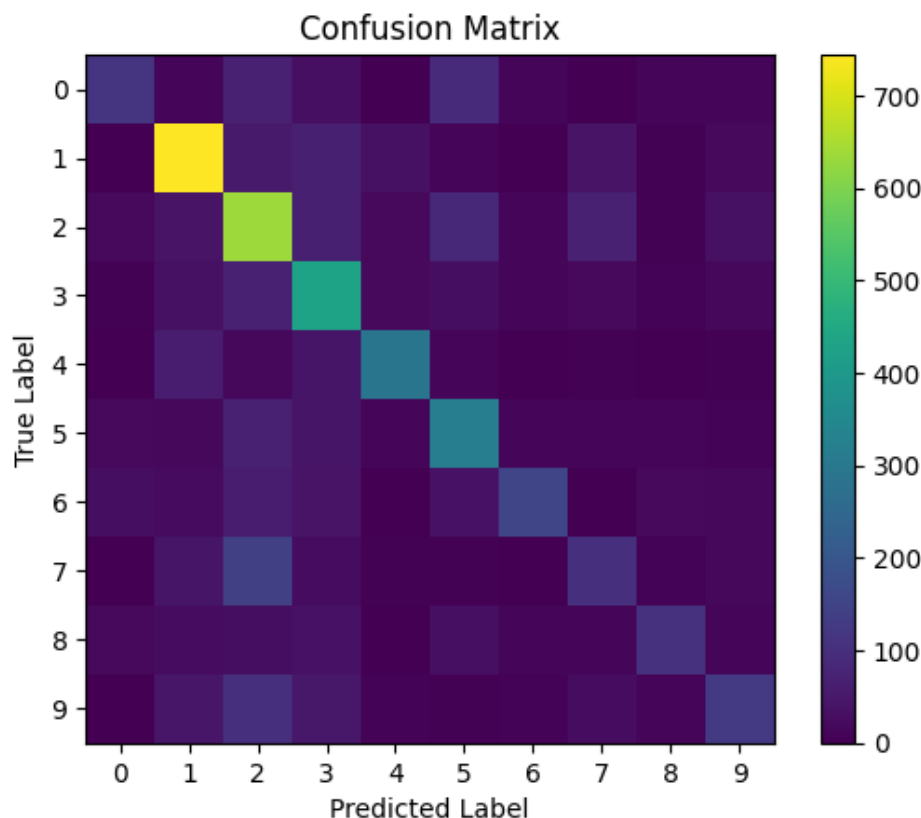
It is the graph of $lr=0.0004$ and $dropout=0.8$ with 0.5773 test accuracy. We can clearly see that epoch 37 is the break of val-loss, it is the minimum here. And we stop there to handle overfitting.



This graph is from $lr=0.0004$ and no-dropout with 0.5023 test accuracy. It is more clear to see the break point.



Here is our best test accuracy convolution matrix. Clearly, we can see the best predictions in classes 1 and 2.



3 Part 2 - Transfer Learning with EfficientNet

3.1 Question 1

Here's why fine-tuning is a valuable approach:

Efficiency: Training a deep learning model from scratch can be extremely time-consuming and require significant computational resources. Fine-tuning leverages the pre-trained model's knowledge, significantly reducing training time and computational cost.

Improved Performance: Pre-trained models often have strong feature extraction capabilities due to their training on vast datasets. Fine-tuning these features for your specific task can lead to better performance compared to training a model entirely from scratch on your smaller dataset.

Data Efficiency: Deep learning models typically require a lot of data to train effectively. Fine-tuning allows you to achieve good results even with a limited amount of data for your specific task.

Why Freeze and Train Only FC Layers?

There are two main reasons for freezing the earlier layers and only training the final fully-connected layers:

Feature Extraction Power: The earlier layers in a pre-trained model have learned general-purpose feature representations that are applicable to many tasks. Freezing these layers prevents them from being overfit to your specific dataset, allowing them to retain their valuable generic feature extraction capabilities.

Focus on Task-Specific Learning: By fine-tuning only the final layers, you direct the model's learning towards the specific details and patterns relevant to your task. This helps the model specialize in solving your problem effectively.

3.2 Question 2

```
1
2 model = torchvision.models.efficientnet_b0(pretrained=True)
3 for param in model.parameters():
4     param.requires_grad_(False)
5 model.classifier[1] = nn.Linear(1280, len(classes))
6 for param in model.classifier[1].parameters():
7     param.requires_grad_(True)
8 # for param in model.features[6].parameters():
9 #     param.requires_grad_(True)
10 # for param in model.features[7].parameters():
11 #     param.requires_grad_(True)
12
13 model.to(device)
14 optimizer = torch.optim.SGD(model.parameters(), lr=0.0004, momentum=0.9)
15 criterion = nn.CrossEntropyLoss()
16 early_stopping = 5
17 train(model, train_loader, device, criterion, optimizer, 100, val_loader,
        early_stopping)
```

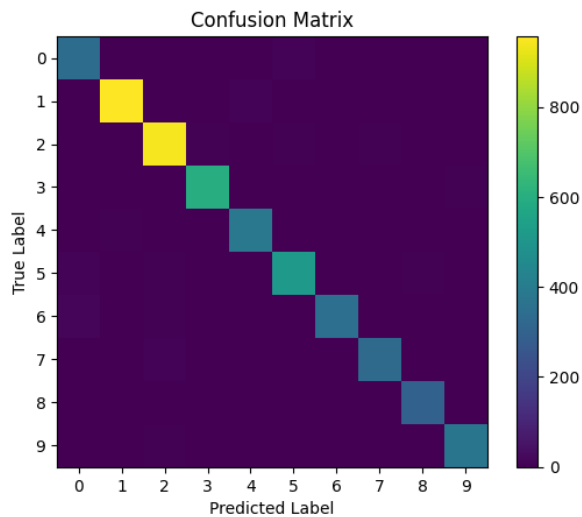
efficientnet optimizer = torch.optim.SGD(model.parameters(), lr=0.0004, momentum=0.9) criterion = nn.CrossEntropyLoss() early-stopping = 5 Val Accuracy: 0.9622 Test Accuracy: 0.9568

```
1
2 model = torchvision.models.efficientnet_b0(pretrained=True)
3 for param in model.parameters():
4     param.requires_grad_(False)
5 model.classifier[1] = nn.Linear(1280, len(classes))
6 for param in model.classifier[1].parameters():
7     param.requires_grad_(True)
8 for param in model.features[6].parameters():
9     param.requires_grad_(True)
10 for param in model.features[7].parameters():
11     param.requires_grad_(True)
12
13 model.to(device)
14 optimizer = torch.optim.SGD(model.parameters(), lr=0.0004, momentum=0.9)
15 criterion = nn.CrossEntropyLoss()
16 early_stopping = 5
17 train(model, train_loader, device, criterion, optimizer, 100, val_loader,
        early_stopping)
```

efficientnet optimizer = torch.optim.SGD(model.parameters(), lr=0.0004, momentum=0.9) criterion = nn.CrossEntropyLoss() early-stopping = 5 Val Accuracy: 0.9752 Test Accuracy: 0.9738

Both of them have good accuracies but the second one is better. Because we trained more layer than the first one and specialized it. But we cannot say that more layer is good every time. If we use too many layers to train more, we may cause to overfit our model.

3.3 Question 3



As it has 0.97 accuracy. The confusion matrix is simple.

3.4 Question 4

Transfer Learning is better with huge difference at accuracy. The causes of this situation: 1- These pretrained models are trained with huge datasets and more calculated layers. So while we are fine-tuning it, we use the advantages of the huge trained model.